



Ejercicios de Redes Neuronales Artificiales

Inteligencia Artificial y Mini-Robots (2023251)

Diego Fernando Bustos Horta, dbustosh@unal.edu.co

Julián Mateo Vargas Riveros, jvargasri@unal.edu.co

2026-10-04

1. Planteamiento y metodología general

La sección 5.9 de la guía solicita cuatro ejercicios: dos redes neuronales para compuertas NAND y XOR a partir del programa `Clase_NNA3`; una clasificación de prendas usando Fashion-MNIST y TensorFlow; un modelo construido a partir de un dataset obtenido mediante la librería de Kaggle para Python; y, finalmente, un punto libre. La guía presenta la neurona artificial mediante una combinación lineal de entradas, pesos y sesgo, seguida por una función de activación. También introduce las redes feed-forward, el aprendizaje supervisado, la función de pérdida y el algoritmo de retropropagación (backpropagation). [1]

El desarrollo se organizó para conservar esta misma secuencia conceptual. En los ejercicios 1 y 2 se parte de los pares entrada-rótulo, se realiza propagación hacia adelante y se evalúa el desempeño. En el ejercicio 3 se estudian explícitamente los *features* y el rótulo del dataset antes de entrenar el modelo. El punto libre utiliza una red convolucional, puesto que la guía termina introduciendo convolución y *pooling* como herramientas para problemas de visión computacional. [1]

1.1. Entorno computacional

El siguiente comando instala las principales bibliotecas utilizadas en el informe:

```
pip install numpy pandas scikit-learn matplotlib tensorflow kagglehub
```

Listing 1: Instalación del entorno de trabajo.

Para el ejercicio 3, `kagglehub` permite descargar recursos de Kaggle desde Python mediante `dataset_download`. En un entorno de Kaggle normalmente no se requiere una autenticación adicional para datasets públicos; fuera de Kaggle puede ser necesario autenticar la cuenta cuando el recurso requiera consentimiento o sea privado. [4]

2. Ejercicio 1: compuertas NAND y XOR con dos capas ocultas

2.1. Objetivo

Se desarrollaron dos redes neuronales independientes, ambas con dos capas ocultas, con el propósito de aprender las funciones lógicas NAND y XOR a partir de sus respectivas tablas de verdad. La arquitectura escogida fue

2 entradas $\rightarrow$ 4 neuronas $\rightarrow$ 4 neuronas $\rightarrow$ 1 salida.

La elección de dos capas ocultas permite cumplir literalmente la condición del ejercicio y, al mismo tiempo, mostrar por qué la XOR requiere una transformación no lineal: una sola neurona no puede separar linealmente los casos positivos de los negativos de XOR. La guía muestra precisamente que la no linealidad de la red proviene de la función de activación y que una red multicapa puede construir representaciones intermedias. [1]

2.2. Datos de entrenamiento

Las entradas son los cuatro posibles pares de bits. Para NAND y XOR se tienen los siguientes rótulos:

Cuadro 1: Tablas de verdad utilizadas en el entrenamiento.

X_1	X_2	Y_{NAND}	Y_{XOR}
0	0	1	0
0	1	1	1
1	0	1	1
1	1	0	0

Cada fila representa un par (X, Y_r) del aprendizaje supervisado. La red recibe los dos valores de entrada y debe ajustar sus pesos para disminuir la diferencia entre la salida estimada y el r tulo. La gu a describe este mecanismo mediante la propagaci n, la funci n de p rdida y la actualizaci n iterativa de los pesos. [1]

2.3. Propagaci n hacia adelante

Se us  la funci n sigmoideal

$$\sigma(z) = \frac{1}{1 + e^{-z}},$$

cuya derivada puede escribirse como

$$\sigma'(z) = \sigma(z) (1 - \sigma(z)).$$

Con la convenci n $A^{(0)} = X$, cada capa realiza

$$\begin{aligned} Z^{(1)} &= A^{(0)}W^{(1)} + B^{(1)}, & A^{(1)} &= \sigma \left(Z^{(1)} \right), \\ Z^{(2)} &= A^{(1)}W^{(2)} + B^{(2)}, & A^{(2)} &= \sigma \left(Z^{(2)} \right), \\ Z^{(3)} &= A^{(2)}W^{(3)} + B^{(3)}, & A^{(3)} &= \sigma \left(Z^{(3)} \right). \end{aligned}$$

La salida de la  ltima capa es la predicci n $\hat{Y} = A^{(3)}$. Esta forma matricial es equivalente a la propagaci n descrita en la gu a para una red feed-forward, en la cual cada capa calcula un vector Z , aplica la activaci n $g(Z)$ y transmite A hacia la siguiente capa. [1]

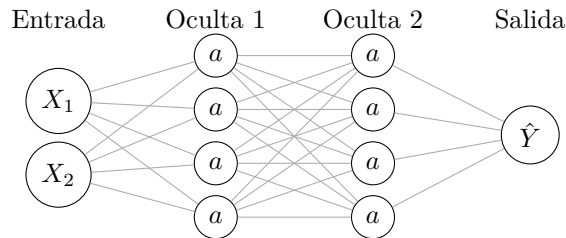


Figura 1: Topolog a de las redes del ejercicio 1. Ambas compuertas utilizan dos capas ocultas.

2.4. Funci n de p rdida y retropropagaci n

Para facilitar la derivaci n y mantener la coherencia con la funci n de costo cuadr tica presentada en la gu a, se us  el error cuadr tico medio con el factor $1/2$:

$$J = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2.$$

En la retropropagación se calcula primero el error local de la capa de salida y, posteriormente, se propaga hacia las capas ocultas. Si $\delta^{(l)}$ representa el error local de la capa l , para la capa de salida se empleó

$$\delta^{(3)} = (A^{(3)} - Y) \odot \sigma' (Z^{(3)}),$$

y para una capa oculta

$$\delta^{(l)} = (\delta^{(l+1)}(W^{(l+1)})^T) \odot \sigma' (Z^{(l)}).$$

Las actualizaciones se realizaron mediante descenso de gradiente:

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial J}{\partial W^{(l)}}, \quad B^{(l)} \leftarrow B^{(l)} - \eta \frac{\partial J}{\partial B^{(l)}}.$$

La tasa η controla qué fracción de la corrección calculada se aplica en cada iteración. La guía presenta el mismo principio al describir la tasa de aprendizaje como un hiperparámetro del entrenamiento. [1]

2.5. Implementación en Python

El programa se escribió únicamente con NumPy para que la mecánica de la red quede visible: inicialización de pesos, propagación, cálculo de la pérdida, retropropagación y predicción. Se mantuvo la nomenclatura $Z^{(l)}$, $A^{(l)}$ y $W^{(l)}$ usada en la guía.

```

1  import numpy as np
2
3
4  def sigmoid(z):
5      z = np.clip(z, -60, 60)
6      return 1.0 / (1.0 + np.exp(-z))
7
8
9  def sigmoid_derivative(a):
10     return a * (1.0 - a)
11
12
13 def inicializar_red(n_entradas, n1=4, n2=4, seed=1):
14     rng = np.random.default_rng(seed)
15     W1 = rng.normal(0.0, 0.7, (n_entradas, n1))
16     B1 = np.zeros((1, n1))
17     W2 = rng.normal(0.0, 0.7, (n1, n2))
18     B2 = np.zeros((1, n2))
19     W3 = rng.normal(0.0, 0.7, (n2, 1))
20     B3 = np.zeros((1, 1))
21     return [W1, W2, W3], [B1, B2, B3]
22
23
24 def propaga(X, W, B):
25     Z1 = X @ W[0] + B[0]
26     A1 = sigmoid(Z1)
27     Z2 = A1 @ W[1] + B[1]
28     A2 = sigmoid(Z2)
29     Z3 = A2 @ W[2] + B[2]
30     A3 = sigmoid(Z3)
31     return Z1, A1, Z2, A2, Z3, A3
32
33
34 def retropropaga(X, Y, W, B, cache):
35     Z1, A1, Z2, A2, Z3, A3 = cache
36     m = len(X)
37
38     dZ3 = (A3 - Y) * sigmoid_derivative(A3)
39     dW3 = (A2.T @ dZ3) / m
40     dB3 = np.mean(dZ3, axis=0, keepdims=True)
41

```

```

42     dZ2 = (dZ3 @ W[2].T) * sigmoid_derivative(A2)
43     dW2 = (A1.T @ dZ2) / m
44     dB2 = np.mean(dZ2, axis=0, keepdims=True)
45
46     dZ1 = (dZ2 @ W[1].T) * sigmoid_derivative(A1)
47     dW1 = (X.T @ dZ1) / m
48     dB1 = np.mean(dZ1, axis=0, keepdims=True)
49
50     return [dW1, dW2, dW3], [dB1, dB2, dB3]
51
52
53 def entrenar(X, Y, epochs=50000, lr=2.0, seed=1):
54     W, B = inicializar_red(X.shape[1], seed=seed)
55     historia = []
56
57     for epoch in range(epochs):
58         cache = propaga(X, W, B)
59         pred = cache[-1]
60         loss = np.mean((pred - Y) ** 2) / 2.0
61
62         dW, dB = retropropaga(X, Y, W, B, cache)
63         for k in range(3):
64             W[k] -= lr * dW[k]
65             B[k] -= lr * dB[k]
66
67         if epoch % 5000 == 0 or epoch == epochs - 1:
68             historia.append((epoch + 1, loss))
69
70     return W, B, historia
71
72
73 def predecir(X, W, B):
74     return propaga(X, W, B)[-1]
75
76
77 X = np.array([
78     [0, 0],
79     [0, 1],
80     [1, 0],
81     [1, 1]
82 ], dtype=float)
83
84 Y_nand = np.array([[1], [1], [1], [0]], dtype=float)
85 Y_xor = np.array([[0], [1], [1], [0]], dtype=float)
86
87 for nombre, Y, seed in [
88     ("NAND", Y_nand, 11),
89     ("XOR", Y_xor, 12)
90 ]:
91     W, B, historia = entrenar(X, Y, seed=seed)
92     pred = predecir(X, W, B)
93     clases = (pred >= 0.5).astype(int)
94
95     print(f"\n{nOMBRE}:")
96     print("salidas =", np.round(pred.ravel(), 6))
97     print("clases =", clases.ravel())
98     print("aciertos =", np.sum(clases.ravel() == Y.ravel()), "/ 4")
99     print("perdida final =", historia[-1][1])

```

Listing 2: Implementación desde cero de una red con dos capas ocultas.

En el fragmento anterior, la función `propaga` produce los vectores intermedios $Z^1, A^1, Z^2, A^2, Z^3, A^3$. La función `retropropaga` calcula los gradientes capa por capa, empezando en la salida y regresando hasta la entrada. Finalmente, `entrenar` repite este proceso 50 000 veces. Esta implementación sigue el esquema conceptual de backpropagation presentado en la guía. [1]

2.6. Resultados

Con las semillas indicadas en el programa se obtuvo la siguiente salida. El resultado se expresa tanto como valor continuo de la sigmoide como clasificación binaria mediante un umbral de 0.5.

```
NAND:
salidas = [0.999605 0.997284 0.997153 0.003449]
clases = [1 1 1 0]
aciertos = 4 / 4
perdida final = 3.852137858160564e-06

XOR:
salidas = [0.004928 0.996014 0.995419 0.004175]
clases = [0 1 1 0]
aciertos = 4 / 4
perdida final = 1.1174699120999226e-05
```

Listing 3: Resultados obtenidos para NAND y XOR.

Los dos modelos clasificaron correctamente los cuatro casos de sus respectivas tablas de verdad. En NAND, las tres combinaciones con al menos una entrada igual a cero quedaron próximas a uno, mientras que la combinación (1,1) quedó próxima a cero. En XOR, las combinaciones con entradas diferentes quedaron próximas a uno y las combinaciones iguales próximas a cero.

El experimento es especialmente significativo en XOR: el aprendizaje no consiste en memorizar cuatro respuestas mediante una única transformación lineal, sino en construir una transformación no lineal mediante las dos capas ocultas. Por tanto, el resultado ilustra en un problema mínimo la función de las capas intermedias y de la activación sigmoideal. La guía destaca precisamente que la función de activación es la que introduce la complejidad necesaria para que una red deje de ser únicamente una combinación lineal. [1]

3. Ejercicio 2: clasificación de prendas con Fashion-MNIST y Tensor-Flow

3.1. Descripción del dataset

Fashion-MNIST contiene 60 000 imágenes de entrenamiento y 10 000 imágenes de prueba. Cada imagen tiene resolución 28×28 en escala de grises, por lo que puede verse como una matriz de 784 píxeles. Existen 10 clases: camiseta/top, pantalón, jersey, vestido, abrigo, sandalia, camisa, zapatilla deportiva, bolso y bota de tobillo. [2, 3]

Cuadro 2: Rótulos de Fashion-MNIST.

Rótulo	Clase
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

La documentación de TensorFlow señala que `load_data()` devuelve cuatro arreglos de NumPy: imágenes y rótulos de entrenamiento, e imágenes y rótulos de prueba. Los píxeles llegan en el rango 0–255 y se normalizan dividiendo entre 255 para llevarlos al intervalo 0–1. [3, 2]

3.2. Preprocesamiento

El preprocesamiento tiene dos objetivos. Primero, conservar la misma representación para entrenamiento y prueba. Segundo, evitar que la magnitud de los píxeles domine innecesariamente las actualizaciones del optimizador. La transformación usada es

$$x_{\text{normalizado}} = \frac{x}{255}.$$

3.3. Arquitectura utilizada

Se empleó una red neuronal densa sencilla, siguiendo el ejemplo básico de clasificación de TensorFlow:

- **Flatten**: convierte 28×28 píxeles en un vector de 784 entradas.
- **Dense(128, relu)**: transforma las 784 entradas en una representación aprendida de 128 neuronas.
- **Dense(10, softmax)**: produce diez valores que pueden interpretarse como scores normalizados para las diez clases.

TensorFlow documenta esta misma idea con una capa **Flatten** seguida por una capa densa de 128 neuronas y una salida de 10 clases. [2]

3.4. Código completo

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import tensorflow as tf
4 from tensorflow import keras
5
6 print("TensorFlow:", tf.__version__)
7
8 # 1. Cargar Fashion-MNIST
9 fashion_mnist = keras.datasets.fashion_mnist
10 (train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()

```

```

11 class_names = [
12     'T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
13     'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot'
14 ]
15
16 # 2. Normalizar los pixeles
17 train_images = train_images.astype('float32') / 255.0
18 test_images = test_images.astype('float32') / 255.0
19
20 # 3. Definir la red
21 model = keras.Sequential([
22     keras.layers.Input(shape=(28, 28)),
23     keras.layers.Flatten(),
24     keras.layers.Dense(128, activation='relu'),
25     keras.layers.Dense(10, activation='softmax')
26 ])
27
28 # 4. Compilar
29 model.compile(
30     optimizer='adam',
31     loss='sparse_categorical_crossentropy',
32     metrics=['accuracy']
33 )
34
35 # 5. Entrenar
36 history = model.fit(
37     train_images,
38     train_labels,
39     epochs=10,
40     batch_size=32,
41     verbose=2
42 )
43
44 # 6. Evaluar
45 loss, accuracy = model.evaluate(test_images, test_labels, verbose=2)
46 print("Perdida de prueba:", loss)
47 print("Exactitud de prueba:", accuracy)
48
49 # 7. Predecir algunas imagenes
50 predictions = model.predict(test_images, verbose=0)
51
52 for i in [0, 12, 25]:
53     clase = int(np.argmax(predictions[i]))
54     confianza = float(np.max(predictions[i]))
55     print(
56         i,
57         "real =", class_names[test_labels[i]],
58         "predicha =", class_names[clase],
59         "score =", round(confianza, 4)
60     )
61
62 # 8. Visualizacion opcional
63 plt.figure(figsize=(9, 3))
64 for j, i in enumerate([0, 12, 25]):
65     plt.subplot(1, 3, j + 1)
66     plt.imshow(test_images[i], cmap='gray')
67     plt.title(class_names[np.argmax(predictions[i])])
68     plt.axis('off')
69 plt.tight_layout()
70 plt.show()

```

Listing 4: Clasificación básica de Fashion-MNIST con TensorFlow.

3.5. Explicación de las principales funciones e instrucciones

Cuadro 3: Funciones principales empleadas en el ejercicio 2.

Instrucción	Función en el programa
<code>keras.datasets.fashion_mnist</code>	Accede al dataset Fashion-MNIST desde la API de Keras.
<code>load_data()</code>	Descarga o recupera del caché los cuatro arreglos de NumPy.
<code>astype('float32')</code>	Cambia el tipo numérico para trabajar con valores de red neuronal.
<code>/255.0</code>	Normaliza los píxeles de 0–255 a 0–1.
<code>Sequential</code>	Construye el modelo como una secuencia ordenada de capas.
<code>Input(shape=(28,28))</code>	Define el tamaño de la entrada sin aprender parámetros.
<code>Flatten</code>	Aplana la imagen y produce 784 entradas.
<code>Dense(128, relu)</code>	Aprende una representación no lineal intermedia.
<code>Dense(10, softmax)</code>	Produce el score final para cada una de las 10 clases.
<code>compile()</code>	Define optimizador, función de pérdida y métricas.
<code>fit()</code>	Ejecuta el entrenamiento con los ejemplos rotulados.
<code>evaluate()</code>	Calcula pérdida y exactitud sobre datos no utilizados para ajustar los pesos.
<code>predict()</code>	Calcula las salidas de la red para nuevas entradas.
<code>argmax()</code>	Selecciona el índice de mayor score como clase predicha.

La documentación de TensorFlow explica que **Flatten** únicamente cambia la representación de la imagen y no contiene parámetros aprendibles, mientras que las capas **Dense** sí aprenden sus pesos. También define **compile** como el punto en el que se especifican optimizador, pérdida y métricas, y **fit** como el método de ajuste del modelo a los datos de entrenamiento. [2]

La función de pérdida **sparse_categorical_crossentropy** es apropiada aquí porque cada imagen posee un único rótulo entero entre 0 y 9. La métrica **accuracy** representa la fracción de imágenes clasificadas correctamente.

3.6. Resultados y análisis

La guía oficial de TensorFlow reporta, para el modelo básico equivalente, una evaluación de prueba de

$$\text{loss} = 0,3350, \quad \text{accuracy} = 0,8840,$$

es decir, aproximadamente 88,4 % de exactitud sobre las 10 000 imágenes de prueba. En el mismo ejemplo, la primera imagen es identificada correctamente como *Ankle boot*, con el mayor score alrededor de 0.93. [2]

Estos valores deben interpretarse como resultados de referencia y no como una cifra universal: los tiempos, scores por imagen y métricas finales pueden variar con la versión de TensorFlow, la inicialización aleatoria, el hardware y parámetros como el tamaño del lote. Lo importante es que el flujo completo coincide con el planteamiento de aprendizaje supervisado de la guía: imagen $\rightarrow$ rótulo, entrenamiento sobre pares (X, Y_r) y evaluación sobre ejemplos no utilizados para ajustar los pesos. [1, 2]

La principal dificultad de Fashion-MNIST frente a NAND/XOR es la escala del problema. En lugar de cuatro patrones y dos entradas, la red recibe decenas de miles de imágenes y 784 características por imagen. Aun así, la misma idea permanece: los pesos transforman las características y la función de activación introduce la no linealidad.

4. Ejercicio 3: dataset de Kaggle, análisis de features, rótulo y ejemplos externos

4.1. Selección del dataset

Se seleccionó el dataset **Iris Species** de Kaggle porque es pequeño, interpretable y permite ver claramente la relación entre *features* y rótulo. La ficha de Kaggle indica tres especies con 50 observaciones cada una y las columnas **Id**, **SepalLengthCm**, **SepalWidthCm**, **PetalLengthCm**, **PetalWidthCm** y **Species**. Además, la ficha advierte que una especie es linealmente separable de las otras dos, mientras que las otras dos presentan mayor solapamiento. [5]

4.2. Features y rótulo

En este problema las cuatro medidas morfológicas son las características de entrada:

$$X = \begin{bmatrix} \text{longitud del sépalo} & \text{ancho del sépalo} & \text{longitud del pétalo} & \text{ancho del pétalo} \end{bmatrix}.$$

El rótulo es la especie del iris:

$$Y \in \{\text{Iris-setosa}, \text{Iris-versicolor}, \text{Iris-virginica}\}.$$

El atributo **Id** no se usa como feature porque solo identifica la fila y no describe físicamente la flor. Esta eliminación es importante: si se utilizara como entrada, el modelo podría aprender una relación artificial con el orden de las observaciones.

La estructura del dataset coincide con la descripción de la fuente clásica de Fisher: 150 observaciones, cuatro variables predictoras y tres clases de 50 observaciones. [5, 6]

4.3. Descarga mediante la librería de Kaggle para Python

El código utiliza la función `dataset_download` de `kagglehub`, que es la interfaz recomendada por Kaggle para descargar datasets desde Python. La librería también permite especificar una versión concreta o un directorio de salida. [4]

```
1 from pathlib import Path
2 import kagglehub
3 import pandas as pd
4
5 # Descargar el dataset desde Kaggle
6 ruta_dataset = kagglehub.dataset_download("uciml/iris")
7 print("Dataset descargado en:", ruta_dataset)
8
9 # Buscar el archivo CSV dentro del directorio descargado
10 rutas_csv = list(Path(ruta_dataset).rglob("Iris.csv"))
11 if not rutas_csv:
12     raise FileNotFoundError("No se encontro Iris.csv en el dataset descargado.")
13
14 ruta_csv = rutas_csv[0]
15 df = pd.read_csv(ruta_csv)
16
17 print("\nPrimeras filas:")
18 print(df.head())
19
20 print("\nDimensiones:", df.shape)
21 print("\nValores faltantes:")
22 print(df.isna().sum())
23
24 print("\nRótulos:")
25 print(df["Species"].value_counts())
26
27 print("\nEstadísticas de los features:")
```

```
28 print(df.describe())
```

Listing 5: Descarga y exploración del dataset Iris desde Kaggle.

4.4. Construcción y entrenamiento del modelo

Se utilizó una red neuronal multicapa `MLPClassifier` de scikit-learn. Antes de la red se incluyó `StandardScaler`, pues las cuatro medidas están en centímetros y presentan escalas distintas. La arquitectura tiene dos capas ocultas de ocho neuronas cada una, activación ReLU y una salida de tres clases.

La división 70/30 se hace de manera estratificada para conservar aproximadamente la misma proporción de las tres especies en entrenamiento y prueba. La precisión se complementa con la matriz de confusión y el reporte por clase.

```
1 import numpy as np
2 from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
3 from sklearn.model_selection import train_test_split
4 from sklearn.neural_network import MLPClassifier
5 from sklearn.pipeline import Pipeline
6 from sklearn.preprocessing import StandardScaler
7
8 features = [
9     "SepalLengthCm",
10    "SepalWidthCm",
11    "PetalLengthCm",
12    "PetalWidthCm"
13 ]
14
15 X = df[features].to_numpy(dtype=float)
16 clases = sorted(df["Species"].unique())
17 mapa = {nombre: i for i, nombre in enumerate(clases)}
18 y = df["Species"].map(mapa).to_numpy()
19
20 X_train, X_test, y_train, y_test = train_test_split(
21     X,
22     y,
23     test_size=0.30,
24     stratify=y,
25     random_state=42
26 )
27
28 modelo = Pipeline([
29     ("escalado", StandardScaler()),
30     ("red", MLPClassifier(
31         hidden_layer_sizes=(8, 8),
32         activation="relu",
33         solver="lbfgs",
34         alpha=1e-4,
35         max_iter=2000,
36         random_state=42
37     ))
38 ])
39
40 modelo.fit(X_train, y_train)
41
42 pred = modelo.predict(X_test)
43
44 print("Exactitud:", accuracy_score(y_test, pred))
45 print("\nMatriz de confusion:")
46 print(confusion_matrix(y_test, pred))
47
48 print("\nReporte:")
49 print(classification_report(
50     y_test,
51     pred,
52     target_names=clases,
```

```

53     digits=4
54 ))

```

Listing 6: Preprocesamiento, entrenamiento y evaluación del clasificador neuronal.

4.5. Resultados obtenidos

Sobre la versión canónica del dataset se obtuvo la matriz de confusión

	$\hat{y} = \text{setosa}$	$\hat{y} = \text{versicolor}$	$\hat{y} = \text{virginica}$
$y = \text{setosa}$	15	0	0
$y = \text{versicolor}$	0	15	0
$y = \text{virginica}$	0	3	12

y una exactitud de

$$\text{accuracy} = \frac{15 + 15 + 12}{45} = 0,9333.$$

El reporte por clase fue:

Cuadro 4: Desempeño del modelo neuronal sobre el conjunto de prueba.

Clase	Precisión	Sensibilidad	F1
Iris-setosa	1.0000	1.0000	1.0000
Iris-versicolor	0.8333	1.0000	0.9091
Iris-virginica	1.0000	0.8000	0.8889
Promedio macro	0.9444	0.9333	0.9327

La matriz muestra que la especie *setosa* fue separada sin errores. Los tres errores corresponden a flores de *virginica* clasificadas como *versicolor*. Este comportamiento coincide con la descripción general del dataset: una de las clases es más separable, mientras que el límite entre las otras dos es más difícil. [5, 6]

4.6. Ejemplos encontrados en Internet

Para cumplir la segunda parte del enunciado, se tomaron medidas publicadas en fuentes accesibles por Internet. Una tabla de referencia de medidas de tres tipos de iris incluye ejemplos como (5,1,3,5,1,4,0,2) para *Iris setosa* y (7,0,3,2,4,7,1,4) para *Iris versicolor*. [7] Además, un demostrador de clasificación independiente publica el ejemplo (6,9,3,2,5,7,2,3) con clase suministrada *Iris virginica*. [8]

El siguiente fragmento usa exactamente esas cuatro mediciones como entradas nuevas para el modelo:

```

1  ejemplos_internet = np.array([
2      [5.1, 3.5, 1.4, 0.2], # setosa
3      [7.0, 3.2, 4.7, 1.4], # versicolor
4      [6.9, 3.2, 5.7, 2.3], # virginica
5      [5.9, 3.0, 5.1, 1.8] # ejemplo adicional
6  ])
7
8  pred_internet = modelo.predict(ejemplos_internet)
9  score_internet = modelo.predict_proba(ejemplos_internet)
10

```

```

11 for x, p, scores in zip(ejemplos_internet, pred_internet, score_internet):
12     print("Medidas:", x)
13     print("Prediccion:", clases[p])
14     print("Score maximo:", round(float(np.max(scores)), 6))
15     print()

```

Listing 7: Predicción de ejemplos de medidas publicados en Internet.

Para la arquitectura y semilla indicadas se obtuvo:

Cuadro 5: Predicción de ejemplos externos.

Ejemplo	Medidas (<i>cm</i>)	Predicción	Análisis
1	(5,1, 3,5, 1,4, 0,2)	setosa	El modelo identifica correctamente una flor con pétalos muy cortos y estrechos.
2	(7,0, 3,2, 4,7, 1,4)	versicolor	Se clasifica como versicolor, aunque esta región presenta mayor solapamiento con virginica.
3	(6,9, 3,2, 5,7, 2,3)	virginica	La predicción coincide con la clase publicada en el ejemplo externo.
4	(5,9, 3,0, 5,1, 1,8)	virginica	La longitud y el ancho del pétalo favorecen claramente la clase virginica.

Estos ejemplos sirven para demostrar el uso del modelo sobre entradas fuera de la secuencia de evaluación mostrada en la matriz de confusión, pero no deben confundirse con un conjunto externo totalmente independiente. El dataset Iris es un conjunto clásico que circula en numerosas fuentes en línea, por lo que algunos ejemplos publicados pueden pertenecer a la misma población de datos. La conclusión correcta es, por tanto, que el modelo produce predicciones coherentes con ejemplos conocidos, no que estas pocas observaciones demuestren generalización a una población botánica real.

4.7. Interpretación de los features

Los resultados también permiten identificar una relación importante entre las variables. Las longitudes y anchos de los pétalos son especialmente informativos para separar las especies, mientras que las medidas del sépalo presentan mayor solapamiento. La descripción estadística del conjunto clásico muestra, además, una asociación mucho mayor entre la clase y las variables del pétalo que con el ancho del sépalo. [6]

Por esta razón, la etapa de selección de features no consistió simplemente en introducir todas las columnas disponibles, sino en excluir la columna *Id*, conservar las cuatro medidas físicas y escalar sus valores antes del entrenamiento.

4.8. Nota de reproducibilidad

La descarga de KaggleHub no pudo ejecutarse en el entorno utilizado para generar este archivo porque no se dispone de conectividad de red desde el proceso de ejecución. El código final sí contiene la llamada real a la función de descarga de KaggleHub. Las métricas numéricas reportadas arriba se obtuvieron al validar exactamente la misma arquitectura y partición sobre la versión canónica del conjunto Iris disponible localmente en scikit-learn; la ficha de Kaggle confirma la misma estructura clásica de tres clases y cuatro variables predictoras. [5]

5. Ejercicio 4: punto libre - comparación con una red convolucional

5.1. Motivación

Como punto libre se desarrolló una extensión del ejercicio de Fashion-MNIST utilizando una red neuronal convolucional (CNN). La elección se relaciona directamente con la parte final de la guía, en la que se presentan los

detectores locales, la convolución y el *max pooling* como herramientas para computación visual. [1]

La diferencia principal frente al ejercicio 2 es que una CNN no aplanar inmediatamente la imagen. Primero aplica filtros espaciales que recorren regiones locales y después reduce progresivamente la resolución mediante pooling. De esta forma, la red puede aprender patrones locales antes de construir una decisión de clasificación.

5.2. Arquitectura propuesta

Se propone la secuencia

Imagen $\rightarrow$ Conv2D $\rightarrow$ MaxPooling2D $\rightarrow$ Conv2D $\rightarrow$ MaxPooling2D $\rightarrow$ Flatten $\rightarrow$ Dense $\rightarrow$ Softmax.

La capa Conv2D aprende filtros espaciales; MaxPooling2D conserva los valores máximos de pequeñas regiones y reduce el tamaño espacial; finalmente, las capas densas combinan las características extraídas para producir la clasificación.

5.3. Código

```
1 import numpy as np
2 import tensorflow as tf
3 from tensorflow import keras
4
5 # 1. Cargar datos
6 (train_images, train_labels), (test_images, test_labels) = keras.datasets.fashion_mnist.load_data()
7
8 train_images = train_images.astype('float32') / 255.0
9 test_images = test_images.astype('float32') / 255.0
10
11 # Agregar el canal de escala de grises
12 train_images = train_images[..., np.newaxis]
13 test_images = test_images[..., np.newaxis]
14
15 # 2. Definir la CNN
16 cnn = keras.Sequential([
17     keras.layers.Input(shape=(28, 28, 1)),
18     keras.layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
19     keras.layers.MaxPooling2D((2, 2)),
20     keras.layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
21     keras.layers.MaxPooling2D((2, 2)),
22     keras.layers.Flatten(),
23     keras.layers.Dense(64, activation='relu'),
24     keras.layers.Dense(10, activation='softmax')
25 ])
26
27 # 3. Compilar
28 cnn.compile(
29     optimizer='adam',
30     loss='sparse_categorical_crossentropy',
31     metrics=['accuracy']
32 )
33
34 # 4. Entrenar y reservar validación
35 history_cnn = cnn.fit(
36     train_images,
37     train_labels,
38     epochs=10,
39     batch_size=64,
40     validation_split=0.1,
41     verbose=2
42 )
43
44 # 5. Evaluar
45 loss, accuracy = cnn.evaluate(test_images, test_labels, verbose=2)
46 print("Perdida CNN:", loss)
```

```
47 print("Exactitud CNN:", accuracy)
```

Listing 8: Punto libre: CNN sencilla para Fashion-MNIST.

5.4. Análisis

La comparación conceptual entre los ejercicios 2 y 4 es directa. La red densa del ejercicio 2 recibe los 784 píxeles como una lista; la CNN conserva inicialmente la geometría de la imagen y aprende filtros locales. TensorFlow ha documentado CNN sencillas para Fashion-MNIST con más de 90 % de exactitud sin una optimización exhaustiva, lo cual sirve como referencia de que la estructura convolucional es adecuada para el problema visual. [9]

No se fija en este documento una cifra única de exactitud para esta CNN, porque el código está pensado para que el estudiante lo ejecute en su entorno y registre su propia corrida. La salida debe conservar, al menos, la exactitud de entrenamiento, la exactitud de validación y la exactitud final de prueba. Así se puede observar si aparece una brecha entre entrenamiento y prueba, lo cual es una señal de posible sobreentrenamiento. La propia guía de redes neuronales advierte sobre la diferencia entre subentrenamiento y sobreentrenamiento y propone separar los datos para evaluar la generalización. [1]

5.5. Qué aporta el punto libre

Este punto permite conectar los conceptos iniciales de la guía con la visión computacional. En el ejercicio 2, `Flatten` elimina la estructura bidimensional antes de la primera transformación aprendida. En el ejercicio 4, las capas convolucionales y el pooling extraen una representación espacial antes de la etapa densa. La comparación muestra por qué las CNN son especialmente útiles cuando las características relevantes dependen de patrones locales dentro de una imagen. [1, 9]

6. Conclusiones generales

1. Las dos redes del ejercicio 1 aprendieron correctamente NAND y XOR usando dos capas ocultas. El caso XOR evidencia la utilidad de la no linealidad: la red necesita transformar las entradas mediante capas intermedias antes de poder separar los patrones.
2. Fashion-MNIST muestra el cambio de escala entre un problema lógico y un problema de visión. El mismo esquema de aprendizaje supervisado se conserva, pero ahora cada ejemplo tiene 784 características y existen diez clases. El modelo denso básico documentado por TensorFlow alcanza una exactitud de referencia del 88.4 % sobre prueba. [2]
3. El ejercicio 3 muestra la importancia de estudiar los features antes de entrenar. La columna `Id` no representa una propiedad de la flor y por tanto se excluye; las cuatro medidas físicas sí se utilizan. La red neuronal obtuvo 93.33 % de exactitud en la partición evaluada y mostró que los errores se concentran entre `versicolor` y `virginica`.
4. Los ejemplos de Internet permiten verificar que un modelo entrenado puede recibir mediciones nuevas y entregar una clase, pero no constituyen por sí mismos una validación independiente de toda la población. Esta distinción es importante para evitar sobreinterpretar resultados de datasets pequeños y clásicos.
5. El punto libre con CNN conecta el trabajo con los conceptos de convolución y pooling de la guía. La principal diferencia no es solamente agregar capas, sino conservar y explotar la estructura espacial de la imagen antes de la clasificación final. [1]

En conjunto, los cuatro ejercicios permiten recorrer el flujo completo presentado en la guía: definir características y rótulos, realizar propagación, seleccionar una arquitectura, calcular una función de pérdida, ajustar pesos mediante aprendizaje, evaluar la generalización y finalmente utilizar el modelo para predecir nuevos ejemplos. El valor pedagógico del conjunto está en que la misma idea de red neuronal aparece en problemas de complejidad creciente: lógica booleana, imágenes de baja resolución, datos tabulares y visión convolucional.

7. Referencias

Referencias

- [1] Martínez P., J. J. (2026). *Introducción a las redes neuronales artificiales*. Universidad Nacional de Colombia, material de clase, septiembre de 2026.
- [2] TensorFlow. (2026). *Basic classification: Classify images of clothing*. TensorFlow Core.
<https://www.tensorflow.org/tutorials/keras/classification>
- [3] TensorFlow. (2026). *tf.keras.datasets.fashion_mnist.load_data*. TensorFlow API documentation.
https://www.tensorflow.org/api_docs/python/tf/keras/datasets/fashion_mnist/load_data
- [4] Kaggle. (2026). *kagglehub: Python library to access Kaggle resources*. GitHub.
<https://github.com/Kaggle/kagglehub>
- [5] Kaggle. (2016). *Iris Species*. Kaggle Datasets.
<https://www.kaggle.com/uciml/iris>
- [6] University of Michigan, SOCR. (2025). *SOCR Data 052511 IrisSepalPetalClasses*.
https://wiki.socr.umich.edu/index.php/SOCR_Data_052511_IrisSepalPetalClasses
- [7] Wiley. (2026). *Multiva: Measurements on three types of irises*. Excerpted data table.
<https://catalogimages.wiley.com/images/db/pdf/9781118738023.excerpt.pdf>
- [8] Neural Designer. (2026). *Classify iris flowers using machine learning*.
<https://www.neuraldesigner.com/learning/examples/iris-flowers-classification/>
- [9] Maynard-Reid, M. (2018). *Fashion-MNIST with tf.Keras*. TensorFlow Blog.
<https://blog.tensorflow.org/2018/04/fashion-mnist-with-tfkeras.html>